



PREREQUISITES

Get Ready to Secure Local AI Agents

A fast refresher on the CLI, Python, Docker, and the LLM + agent security mindset, everything you need before Week 1.

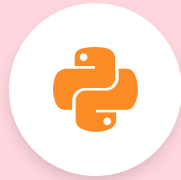
What you'll be ready for

Four areas, kept light where you're already strong. Security and LLM basics are a refresher; setup is hands-on.



CLI & Setup

Shell basics, env vars, getting your machine course-ready.



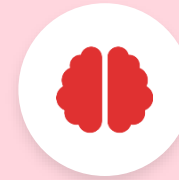
Python for Agents

Read & edit agent code: tools, decorators, f-strings, async.



Docker

Run the labs in identical containers; reach host Ollama.



LLM & Agent Mindset

Tokens, tool-calling, RAG, MCP, and the one security idea.

Session 1 CLI · Python · Docker (get set up)

Session 2 LLM & agent mindset · security framing · readiness check



THE SETUP RUNBOOK

Copy-paste, top to bottom. Green checks at the end = you're ready.

STEP 1

Install the three core tools



Python 3.11+

Runs the agent code and labs.

```
python3 --version  
# need 3.11+  
  
pip3 --version
```



Docker

Runs each week's lab identically.

```
docker run --rm \  
  hello-world  
  
docker compose version
```



Ollama

Serves local models over HTTP.

```
ollama --version  
  
ollama serve &  
# macOS app auto-starts
```



Why these three

Python runs the labs · **Docker** guarantees everyone's environment is identical regardless of OS or hardware · **Ollama** runs the models locally and auto-decides GPU vs CPU, so the same lab works on every tier.

STEP 2

Pull your tier's models

Ollama auto-detects your hardware. You only choose which model string to pull — nothing else in the course changes.

Role	Tier A · 24GB GPU	Tier B · 8–16GB GPU	Tier C · CPU only
Orchestrator	qwen2.5:14b	qwen2.5:7b	qwen2.5:3b
Specialist	qwen2.5:7b	llama3.2:3b	llama3.2:3b
Attacker	llama3.2:3b	llama3.2:1b	llama3.2:1b
Guardrail	llama-guard3:8b	llama-guard3:1b	llama-guard3:1b
Embeddings	nomic-embed-text	nomic-embed-text	all-minilm

Search models on Ollama.com.

<https://ollama.com/search?q=qwen2.5>

Tier C example (CPU)

```
ollama pull qwen2.5:3b
ollama pull llama3.2:3b
ollama pull llama3.2:1b
ollama pull all-minilm
ollama pull \
  llama-guard3:1b

ollama list
```



Don't know your VRAM? You don't need to. Ollama runs the model on GPU, CPU, or a split — whichever fits. If a model is too big, drop a tier. Only **speed** differs across tiers — never the lessons.

STEP 3

Verify, make your repo, smoke-test

Confirm Ollama answers over HTTP

```
curl http://localhost:11434/api/tags

curl localhost:11434/api/generate \
  -d '{"model":"qwen2.5:3b",
      "prompt":"hi","stream":false}'
```

Create your portfolio repo (fill in weekly)

```
mkdir -p secure-multi-agent-lab/\
  {week1,week2,week3,week4,week5,\
    week6,teaching-materials,shared}
cd secure-multi-agent-lab && git init
```

Models live in host Ollama; your repo holds your work.



The smoke test (the gate)

```
cd secure-agents-week1
ORCHESTRATOR_MODEL=qwen2.5:3b \
  docker compose run --rm \
  agent python check_env.py
```

Expected output

- ✓ Ollama reachable
- ✓ model 'qwen2.5:3b' responded
- ✓ Phoenix tracing up
- ✓ **ready for Week 1**

All green? You're ready. Otherwise → Troubleshooting.

PYTHON FOR AGENTS

You don't need to be an expert, just to read, edit, and run agent code.

The constructs you'll see in every lab

```
# venv - isolate dependencies
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# @tool - a typed function the agent can call
from langchain_core.tools import tool

@tool
def get_weather(city: str) -> str:
    """Return weather for a city."""
    # ^ docstring = the tool DESCRIPTION
    return f"{city}: 72F"

# f-string - build a prompt
prompt = f"Summarize: {text}"
```

Not a Dev, No Problem.
VIBE CODE 😊



Decorators (@tool)

Marks a function as an agent tool. The docstring becomes the description the model reads, a security detail in Week 5.



Pydantic models

Typed, validated structures. Used for safe structured outputs from Week 2 on.



Env vars & files

`os.environ.get('ORCHESTRATOR_MODEL')` and `open(path).read()`, every lab uses both.



A little async

`await agent.ainvoke(...)` and `asyncio.run(...)`. MCP calls are async, don't be surprised.



DOCKER

You mostly use containers. You don't author them. Know enough to run labs and reach host Ollama.

Run the labs, reach the models



Image = blueprint · Container = running instance

```
docker pull ...week1:latest    # get image
docker images                  # list
docker ps                      # running
docker compose up              # start lab
docker compose down            # stop & clean
```



The #1 snag: container → host Ollama

Models run in host Ollama, not in the container — so the container must reach OUT to the host.

Mac / Windows: host.docker.internal works automatically.

Linux: compose adds extra_hosts: host-gateway (or use --network=host + localhost).



Read a compose file

```
services:
  phoenix:                # tracing UI, no login
    image: arizephoenix/phoenix:latest
    ports: ["6006:6006"]

  agent:                  # the lab code
    build: .
    environment:
      - ORCHESTRATOR_MODEL=\
        ${ORCHESTRATOR_MODEL:-qwen2.5:3b}
      # ^ the ONE knob you change per tier
      - OLLAMA_HOST=\
        ${OLLAMA_HOST:-\
          http://host.docker.internal:11434}
    extra_hosts:
      - "host.docker.internal:host-gateway"
      # ^ reach host Ollama on Linux too
```



LLM & AGENT MINDSET

Just enough vocabulary. No training, no math. The terms the course assumes.

The working vocabulary



Token & context window

The unit a model reads/writes; the max it can see at once.



Temperature

Randomness. We use 0 for reproducible labs.



Agent (vs chatbot)

An LLM in a loop that calls tools and acts.



RAG & memory

Retrieved docs and persisted facts injected into context.



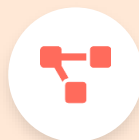
Prompt roles

system / user / assistant / tool messages.



Tool / function calling

Model requests a function; your code runs it. Text → action.



Supervisor + specialists

One agent routes work to others, combines results.



MCP

Agents discover/call external tools; descriptions enter context.



The one idea that frames the whole course

The model does **what the text says**, and “the text” includes user input, retrieved documents, memory, and tool descriptions.

Any of those channels can carry an attacker's instructions.

So security is the architecture AROUND the model, not a property of the model.

allow-listing · input/data separation · validation · provenance · isolation · sandboxing · human-in-the-loop · least privilege

This is “defense in depth”, layered controls so one failure isn't fatal. It's the course's repeated lesson.

Security terms (a quick refresh)

You likely know these, here they are in agent terms.

1

CIA triad

Confidentiality · Integrity · Availability, what we're protecting.

2

Least privilege → least agency

Give a component only what it needs. For agents: autonomy is earned, not default.

3

Trust boundary

Where data crosses from less-trusted to more-trusted. Every agent-reads-another is one.

4

Injection

Untrusted input treated as instructions. Prompt injection = this, for LLM context.

5

Direct vs indirect

Direct = in the user turn. Indirect = hidden in a doc/result the agent reads (worse).

6

Defense in depth

Layered controls so a single failure isn't catastrophic. The whole game.

Readiness checklist

Tick all five and the rest of the course is hands-on.



I can use a terminal, move dirs, and set an env var.



`docker run --rm hello-world` works and I can read a compose file.



`ollama list` shows my models; `curl localhost:11434/api/tags` returns JSON.



I can read a ~40-line Python file with `@tool` functions and follow it.



I can define: tool calling · prompt injection (direct/indirect) · least privilege · trust boundary · defense in depth.



You're ready.

Next up: Week 1 : build your first agent, inject it, and defend it.

Keep the command reference handy. Every command from today lives there, organized by section.